🧠 Desafíos de Lógica, Algoritmos y Estructuras de Datos

Bienvenido a la sección de **Pruebas Técnicas** enfocada en resolver problemas de lógica y algoritmos. Aquí encontrarás ejercicios típicos de entrevistas para practicar habilidades fundamentales de programación, mejorar tu razonamiento y prepararte para pruebas en vivo.

🎯 Qué vas a encontrar aquí

- Enunciados claros y concisos.
- Restricciones y objetivos definidos.
- Soluciones explicadas paso a paso.
- Código de ejemplo en JavaScript (puedes adaptar a tu lenguaje favorito).

💡 **Recomendación**: intenta resolver cada desafío por tu cuenta antes de mirar la solución.

📌 Índice de desafíos

1. ✅ Subcadena más larga sin caracteres repetidos
2. ✅ FizzBuzz con condiciones personalizadas
3. ✅ Verificar si una cadena es palíndromo
4. ✅ Determinar si dos strings son anagramas
5. ✅ Encontrar el número que se repite una sola vez
6. ✅ Validar paréntesis balanceados
7. ✅ Búsqueda binaria en array ordenado
8. ✅ Factorial recursivo vs iterativo

---

1. Subcadena más larga sin caracteres repetidos

1.1 Enunciado

Dado un string, encuentra la longitud de la subcadena más larga que no contiene caracteres repetidos.

1.2 Ejemplo

- **Entrada**: `"abcabcbb"`
- **Salida**: `3` (`"abc"`)

1.3 Explicación paso a paso

1. Usa un puntero izquierdo y derecho para delimitar la ventana.
2. Utiliza un `Set` para registrar caracteres únicos en la ventana.
3. Si hay repetidos, mueve el puntero izquierdo y elimina del set.
4. Guarda el tamaño máximo en cada iteración.

---

2. FizzBuzz con condiciones personalizadas

2.1 Enunciado

Imprime los números del 1 al 100. Para múltiplos de 3, imprime "Fizz", para múltiplos de 5 "Buzz", y para ambos "FizzBuzz".

2.2 Salida esperada

`1, 2, Fizz, 4, Buzz, Fizz, 7, ...`

2.3 Explicación paso a paso

1. Usa un bucle de 1 a 100.
2. Evalúa las condiciones en este orden:

   - `n % 3 === 0 && n % 5 === 0` → "FizzBuzz"
   - `n % 3 === 0` → "Fizz"
   - `n % 5 === 0` → "Buzz"
   - En cualquier otro caso → número

---

3. Verificar si una cadena es palíndromo

3.1 Enunciado

Escribe una función que determine si una palabra o frase es un palíndromo (se lee igual al derecho que al revés).

3.2 Ejemplo

- **Entrada**: `"anita lava la tina"`
- **Salida**: `true`

3.3 Explicación paso a paso

1. Elimina espacios y convierte el texto a minúsculas.
2. Compara la cadena con su versión invertida.

---

4. Determinar si dos strings son anagramas

4.1 Enunciado

Dados dos strings, determina si son anagramas, es decir, si tienen los mismos caracteres en distinto orden.

4.2 Ejemplo

- **Entrada**: `"listen", "silent"`
- **Salida**: `true`

4.3 Explicación paso a paso

1. Quita espacios y normaliza ambas cadenas.
2. Ordena sus caracteres.
3. Compara si el resultado final es idéntico.

---

5. Encontrar el número que se repite una sola vez

5.1 Enunciado

Dado un array donde todos los elementos se repiten dos veces excepto uno, encuentra ese número.

5.2 Ejemplo

- **Entrada**: `[4, 1, 2, 1, 2]`
- **Salida**: `4`

5.3 Explicación paso a paso

1. Usa XOR (`^`) para cancelar los números duplicados.
2. El valor restante será el número único.

---

6. Validar paréntesis balanceados

6.1 Enunciado

Verifica si una cadena de paréntesis está correctamente balanceada.

6.2 Ejemplo

- **Entrada**: `"({[]})"`
- **Salida**: `true`

6.3 Explicación paso a paso

1. Usa una pila.
2. Por cada apertura, agrega el símbolo a la pila.
3. Por cada cierre, compara con el elemento del tope.

---

7. Búsqueda binaria en array ordenado

7.1 Enunciado

Dado un array ordenado y un número objetivo, devuelve su índice o `-1` si no existe.

7.2 Ejemplo

- **Entrada**: `[1, 3, 5, 7, 9]`, objetivo `5`
- **Salida**: `2`

7.3 Explicación paso a paso

1. Usa punteros `inicio` y `fin`.
2. Calcula el punto medio.
3. Reajusta el rango según el valor buscado.

---

8. Factorial recursivo vs iterativo

8.1 Enunciado

Implementa una función que calcule el factorial de un número de estas dos formas:

- Recursiva
- Iterativa

8.2 Ejemplo

- **Entrada**: `5`
- **Salida**: `120`

8.3 Explicación paso a paso

- Recursivo: `n * factorial(n - 1)`
- Iterativo: usar un bucle acumulativo

📌 Cómo practicar

1. Elige un problema del índice.
2. Lee el enunciado sin mirar la solución.
3. Intenta resolverlo por tu cuenta.
4. Revisa la solución paso a paso.
5. Refactorea y optimiza tu código si es necesario.

✅ Puedes adaptar el código a otros lenguajes como Python, Java, C++, etc.

🚀 Nivel de dificultad sugerido

- Desafíos 1 a 5: 🟢 Básico
- Desafíos 6 a 8: 🟡 Intermedio

🛠️ Próximamente

- Nuevos ejercicios por categorías: recursión, sorting, grafos, etc.
- Soluciones en múltiples lenguajes.
- Simulador de entrevistas técnicas.

Hecho con 💻 por [@holasoymalva](https://holasoymalva.com)